

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	KAILASH M	Reg. No.:	192524029
Section / Batch:	2025	Date:	28/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I KAILASH M (192524029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: KAILASH M

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root	Identifies most errors with minor gaps	Identifies some errors but	Unable to identify errors correctly

		causes with clear understanding		misses key issues	
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging Convex Hull Orientation Logic (All Collinear Sets Mishandled in C)

Problem Identification

The bug is in the final line of isHullEdge:

```
return 1; /* ISSUE on fully collinear sets */
```

Walk through what happens when the entire point set is collinear:

- For every point i , $\text{val} = 0$ (no three collinear points ever produce a nonzero cross product).
- Since val is never nonzero, sign never gets set — it stays 0 for the whole loop.

- The else if branch that rejects a bad edge ($\text{sign} < 0$ / $\text{sign} > 0$ conflicts) never triggers either, because it also requires $\text{val} \neq 0$.
- So the function falls through to return 1.

The catch: this doesn't just happen for the correct hull pair — it happens for every possible pair (p_1 , p_2) you test, because any two points from a collinear set still have all other points lying exactly on that same line ($\text{val} == 0$ for all i). The function can't distinguish "these are the true endpoints of the degenerate hull" from "these are two arbitrary interior points on the line." Root cause: the code conflates "no point violates one-sidedness" with "this segment is on the hull boundary," which are only equivalent when the point set is not fully collinear.

Debugging

A collinear point set doesn't have a 2D hull at all — it degenerates to a line segment whose only valid "hull edge" is the one connecting the two extreme points (min and max along the line's direction). Every other pair of points on that line is an interior pair, not a hull edge, and must return 0.

Fix strategy:

- Keep the existing sign-check loop untouched — it's correct for genuine (non-degenerate) hulls.
- Only when the loop finishes with $\text{sign} == 0$ (meaning every point, not just some, is collinear with p_1 – p_2) do we know we're in the degenerate case.
- In that case, resolve it separately: find the two extreme points of the whole set and accept (p_1 , p_2) only if it matches that pair.

Optimization

Calling an $O(n)$ "find extremes" routine inside `isHullEdge` would be correct but wasteful, since `isHullEdge` is itself called $O(n^2)$ times by the brute-force hull driver — recomputing the extremes on every degenerate call adds redundant work. Instead, compute the extremes once, before the $O(n^2)$ pair-testing loop begins, and reuse the result:

```
#include <stdio.h>

/*
 * findExtremes - O(n) helper, called ONCE (not per pair).
 * Projects points onto x (or y, if the set is a vertical line) and
 * records the indices of the minimum and maximum projected point.
 * These are the only two points that can be a valid "edge" when the
 * whole data set is collinear.
 */
void findExtremes(int x[], int y[], int n, int *lo, int *hi) {
    int i, useX = 0;

    for (i = 1; i < n; i++) {
        if (x[i] != x[0]) { useX = 1; break; }
    }

    *lo = *hi = 0;
    for (i = 1; i < n; i++) {
        int val = useX ? x[i] : y[i];
```

```

        int curLo = useX ? x[*lo] : y[*lo];
        int curHi = useX ? x[*hi] : y[*hi];
        if (val < curLo) *lo = i;
        if (val > curHi) *hi = i;
    }
}

/*
 * isHullEdge - Brute-force test for whether (p1, p2) is a convex
 * hull edge: valid if every other point lies on one side of the
 * line through p1 and p2 (or exactly on it).
 *
 * FIX: previously, if every point was collinear with p1 and p2
 * (sign stayed 0), the function returned 1 unconditionally. This
 * made EVERY pair of points look like a hull edge whenever the whole
 * data set was collinear, instead of only the two true endpoints.
 * Now, in that degenerate case, we defer to the precomputed extreme
 * pair (lo, hi) instead of blindly accepting the segment.
 */
int isHullEdge(int p1, int p2, int x[], int y[], int n, int lo, int hi) {
    int i, sign = 0;

    for (i = 0; i < n; i++) {
        int val = (y[p2] - y[p1]) * (x[i] - x[p2])
            - (x[p2] - x[p1]) * (y[i] - y[p2]);

        if (val != 0 && sign == 0) {
            sign = (val > 0) ? 1 : -1;
        } else if ((val > 0 && sign < 0) || (val < 0 && sign > 0)) {
            return 0; /* points on both sides -> not a hull edge */
        }
    }

    if (sign == 0) {
        /* Degenerate case: whole set collinear with p1,p2.
         * Only the precomputed extreme pair counts as a hull edge. */
        return (p1 == lo && p2 == hi) || (p1 == hi && p2 == lo);
    }

    return 1; /* genuine, non-degenerate hull edge */
}

/* Example driver showing correct usage */
int main(void) {
    int x[] = {0, 1, 2, 3, 4};
    int y[] = {0, 1, 2, 3, 4}; /* fully collinear set */
    int n = 5, lo, hi;

    findExtremes(x, y, n, &lo, &hi); /* computed ONCE: O(n) */

    printf("Hull edges:\n");
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isHullEdge(i, j, x, y, n, lo, hi))
                printf("(%d,%d) - (%d,%d)\n", x[i], y[i], x[j], y[j]);
        }
    }
}

```

```
    }  
  }  
  return 0;  
}
```

For this input, the corrected program prints exactly one edge — (0,0)-(4,4) — instead of the original buggy version's 10 (every pair among 5 collinear points).

Analysis

- Original buggy behavior: for a collinear set, every one of the $C(n,2)$ pairs was reported as a hull edge — an $O(n^2)$ -sized "hull," which is nonsensical since a line segment has exactly one edge.
- Root cause, restated simply: the code treated "no contradiction found" as proof of "this is a hull edge," but with an all-zero cross product, there's no information at all to make that call — it needs an independent rule (the extremes) to break the tie.
- Why the fix is correct: in the non-degenerate branch, nothing changed, so correctness for normal point sets is preserved. In the degenerate branch, the only two points that can legitimately bound a set of collinear points are the ones at the extremes of the line — any other pair has points strictly between them, which the previous logic couldn't see because zero cross products carry no directional information.

Time Complexity

- findExtremes: $O(n)$, called once.
- Brute-force hull driver: tests all $C(n,2)$ pairs, each via isHullEdge, which itself is $O(n) \rightarrow O(n^3)$ total, same as the original algorithm.
- The fix adds no asymptotic overhead: findExtremes is $O(n)$ total (not per call), and the degenerate check inside isHullEdge is $O(1)$. So the corrected algorithm keeps the standard $O(n^3)$ brute-force convex hull complexity, while also correctly handling the previously wrong $O(n^2)$ "edges" that the buggy version would have produced on collinear input.